

Clustering Techniques

Juan G. Colonna
jgcolonna@uea.edu.br

Clustering

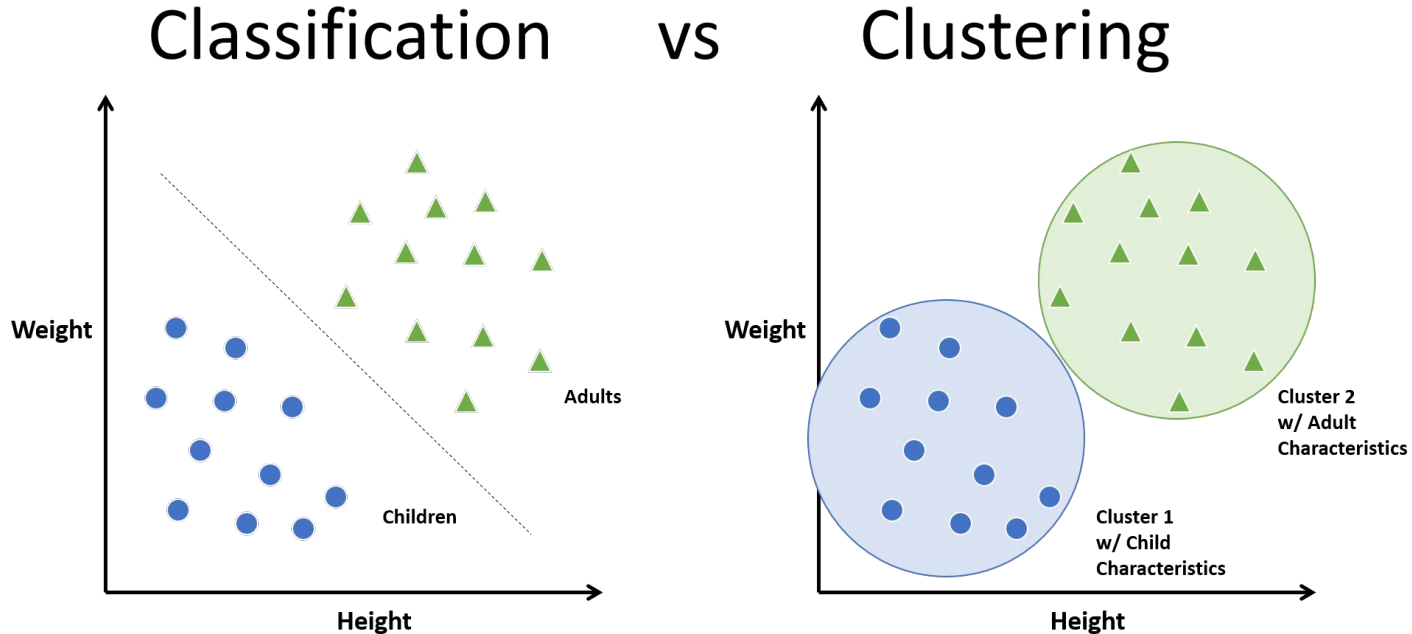
Nesta aula iremos explorar uma categoria de modelos de aprendizado de máquina não supervisionados: algoritmos de agrupamento.

Os algoritmos de agrupamento procuram aprender, a partir dos atributos dos dados, uma divisão ótima ou rotulagem discreta de grupos de pontos.

Muitos algoritmos de clustering estão disponíveis no Scikit-Learn, mas talvez o mais simples de entender seja um algoritmo conhecido como *k*-means, que é implementado em `sklearn.cluster.KMeans`.

Clustering

link: <https://drive.google.com/file/d/1C8a1Dcak9VfF3NYyYgMP1NXnFQPnb3cy/view?usp=sharing>



k -Means

O k significa que o algoritmo procura um número predeterminado de clusters em um conjunto de dados multidimensional não rotulado.

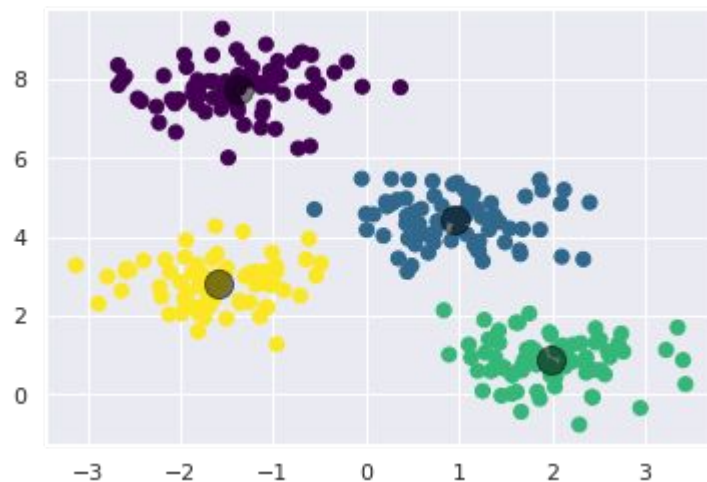
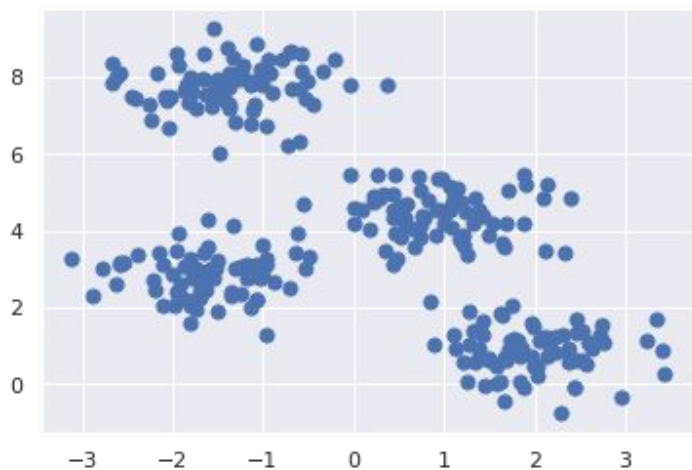
Ele consegue isso usando uma concepção simples de como é o clustering ideal:

- O "centro do cluster" é a média aritmética de todos os pontos pertencentes ao cluster.
- Cada ponto está mais próximo de seu próprio centro de cluster do que de outros centros de cluster.

Essas duas suposições são a base do modelo k -means.

k -Means

Em duas dimensões é fácil verificar os grupos que o k -means encontra:



Algoritmo k -Means (Expectation-Maximization)

Mas como esse algoritmo encontra esses clusters?

K-means utiliza uma abordagem iterativa intuitiva conhecida como Expectation-Maximization (E-M) e consiste no seguinte procedimento:

1. Inicializar aleatoriamente os k -centróides e repetir até convergir:
 - a. E-Step: atribuir pontos do dataset ao centróide mais próximo para formar os clusters; e
 - b. M-Step: redefinir os centróides do cluster para usando a média dos pontos do grupo.

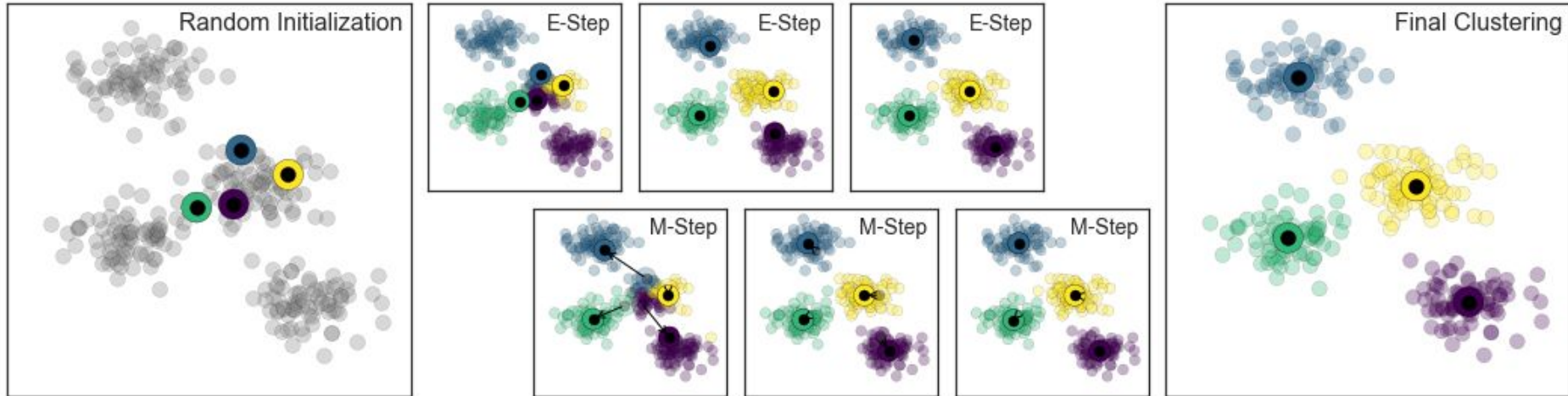
A complexidade no pior caso resulta $O(n(k+2/p))$, onde n = amostras, p = features.

Algoritmo k -Means (Expectation-Maximization)

Aqui, a etapa “E” envolve a atualização de nossa expectativa de qual cluster cada ponto pertence.

A etapa “M” envolve a maximização de alguma função de ajuste, que neste caso, essa maximização é realizada tomando uma média simples dos dados em cada cluster .

Em circunstâncias típicas, cada repetição dos passos E e M sempre resultará em uma estimativa melhor do agrupamento.

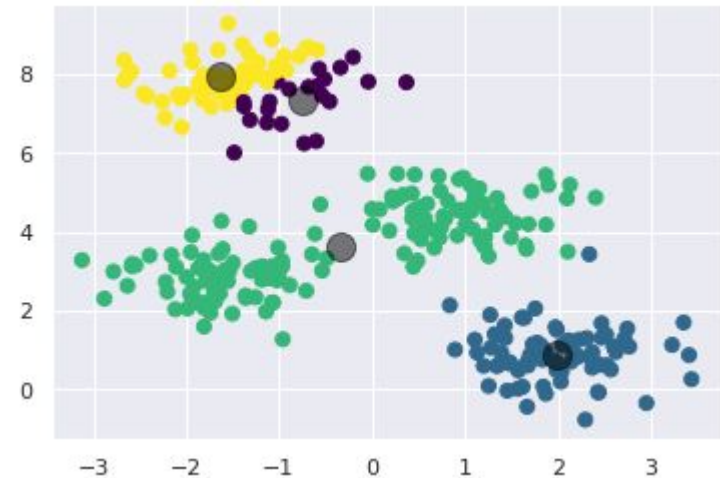


Algumas dificuldades do k-Means

Existem alguns problemas que você deve conhecer antes de usar o algoritmo:

1. Embora seja garantido que o procedimento E-M melhore o resultado em cada iteração, não há garantia de que esta irá convergir para a melhor solução global (ótimo global).

Por exemplo, se usarmos uma semente aleatória diferente para gerar os centróides iniciais, podemos acabar com resultados não tão bons:



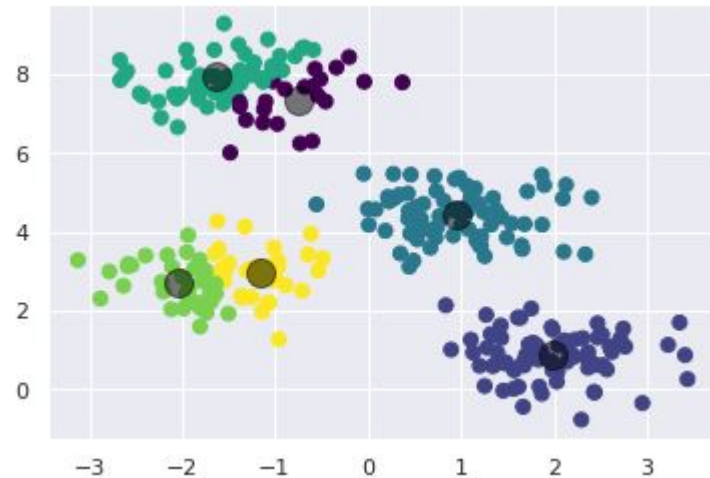
Algumas dificuldades do k-Means

Existem alguns problemas que você deve conhecer antes de usar o algoritmo:

2. O número de clusters deve ser selecionado com antecedência

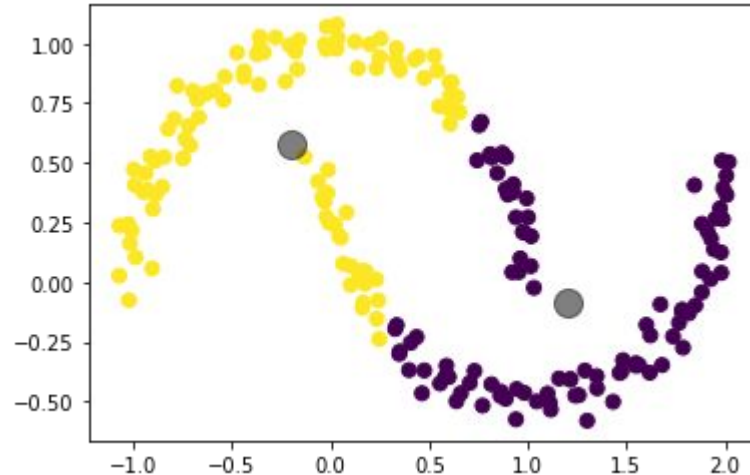
Outro desafio comum é que você deve informar quantos clusters espera encontrar.

Por exemplo, se pedirmos ao algoritmo para identificar seis clusters, ele infelizmente o encontrará:



Algumas dificuldades do k-Means

O limites de separação dos clusters são lineares o que significa que o algoritmo será ineficaz se os clusters tiverem geometrias complicadas.



Algumas dificuldades do k-Means

k-means pode ser lento para um grande número de amostras

Como cada iteração de k-means deve percorrer todos os pontos no conjunto de dados, portanto o algoritmo pode ser relativamente lento conforme o número de amostras aumenta.

Seria possível utilizar apenas usar um subconjunto dos dados para atualizar os centróides do cluster em cada iteração?

Existe uma versão baseada em lotes de dados implementada em `sklearn.cluster.MiniBatchKMeans`.

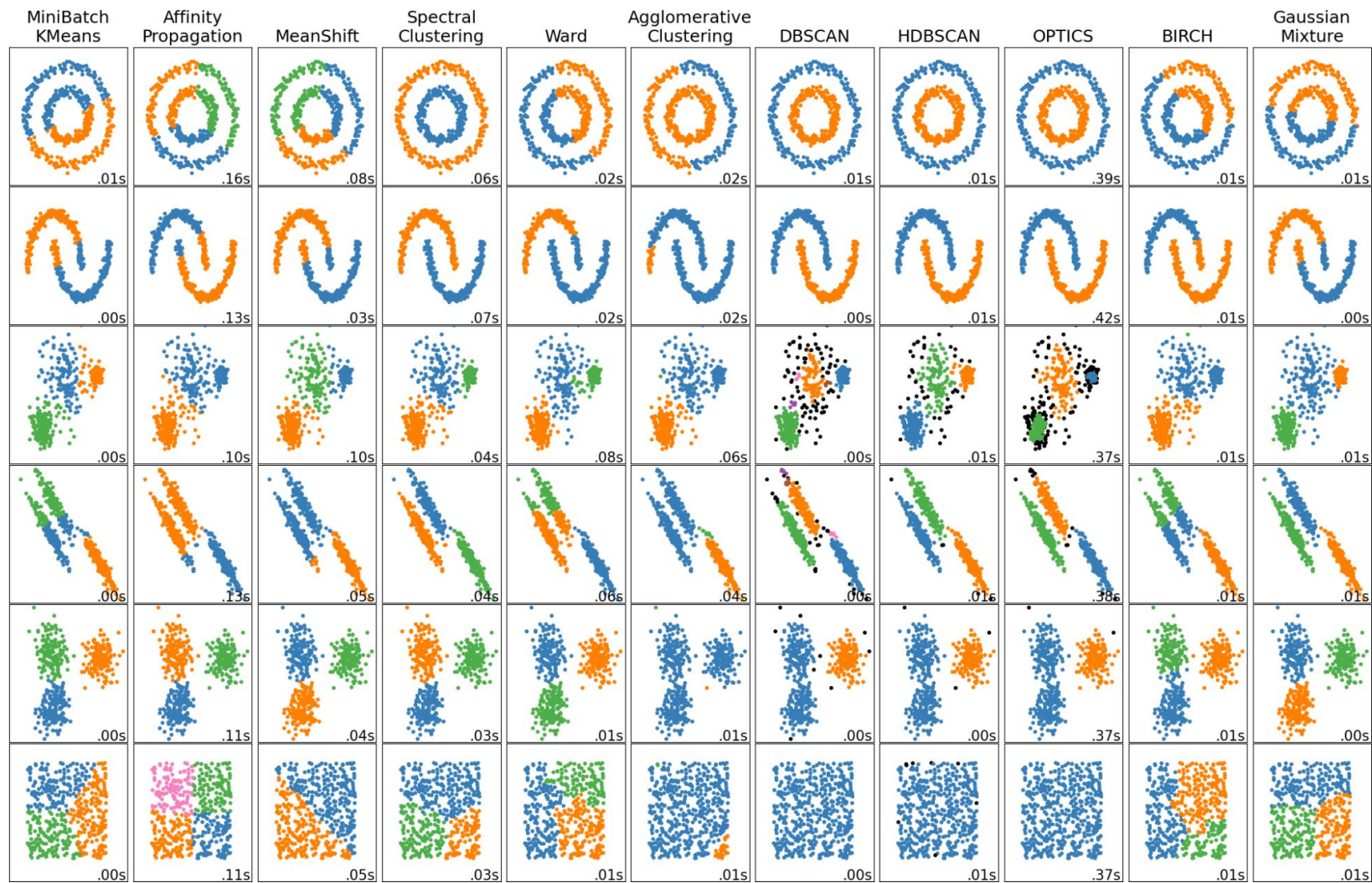
Algumas dificuldades do k-Means

Por último..... Se o resultado é representativo ou não é uma questão difícil de responder.

Uma abordagem intuitiva para observar a qualidade do agrupamento é o gráfico Silhouette (http://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html).

Alternativamente, podemos usar outros algoritmos de agrupamento que possuem outras propriedades:

- Modelos de mistura gaussiana,
- DBSCAN,
- Spectral clustering,
- etc.



Avaliação

Em algoritmos de cluster não temos o ground-truth dos rótulos. Por isso precisamos de algumas métricas, principalmente aquelas que ajudam a escolher o número ótimo de k .

Aqui vamos ver duas:

- Silhouette
- Elbow

Silhouette

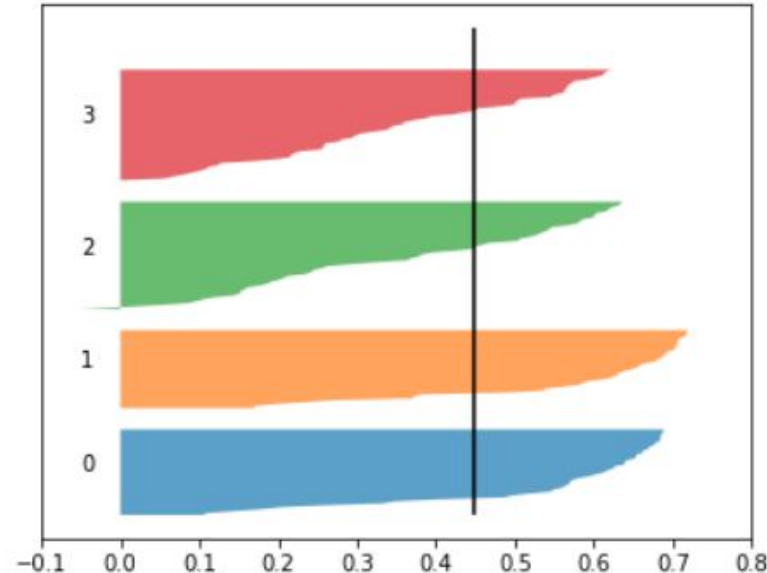
A análise de Silhouette pode ser usada para estudar a distância de separação entre os membros dos clusters e seus centróides.

O score do Silhouette exibe a medida de quão próximo cada ponto em um cluster está do pontos nos clusters vizinhos. Esta medida tem um intervalo de $[-1, 1]$.

A distância de cada amostra pode ser vista no gráfico →

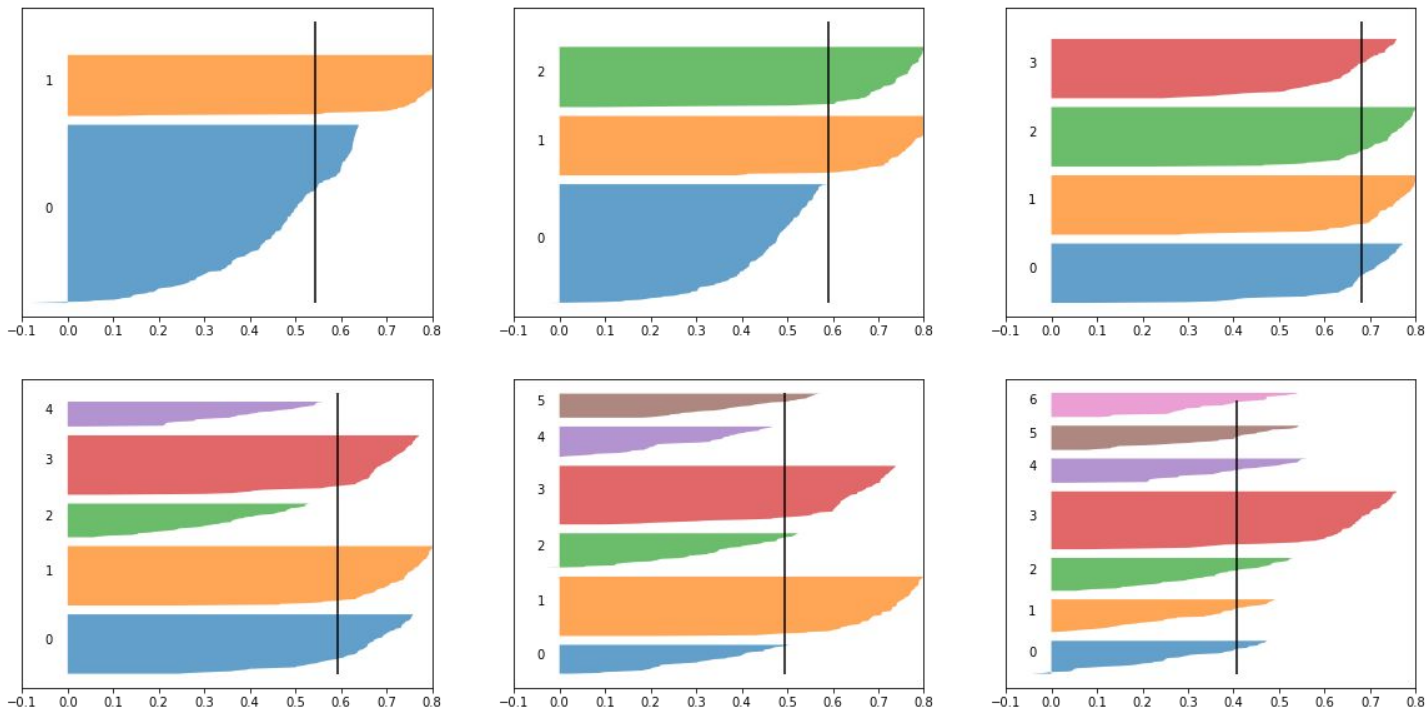
- Valores próximos a +1 indicam que a amostra está longe dos clusters vizinhos;
- Um valor de 0 indica que a amostra está perto do limite de decisão entre dois clusters vizinhos; e
- Valores negativos indicam que essas amostras podem ter sido atribuídas ao cluster errado.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html



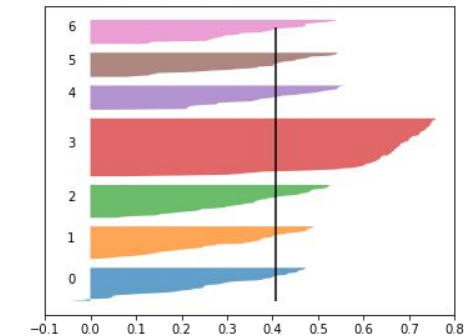
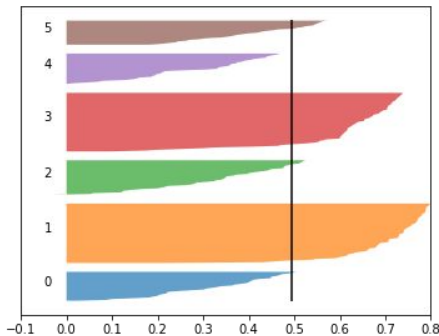
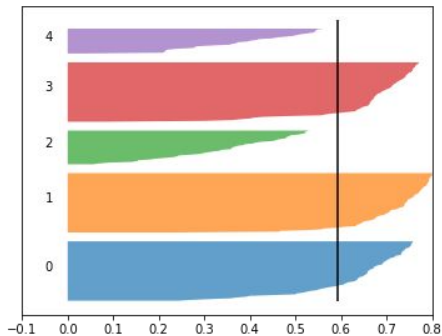
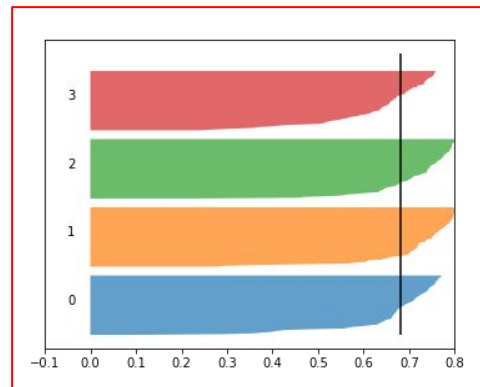
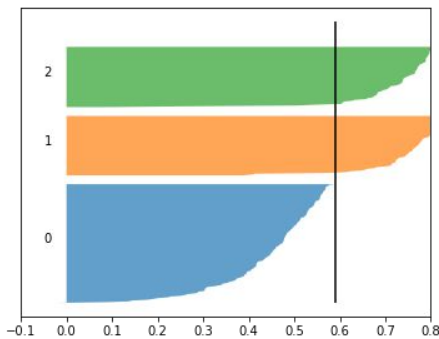
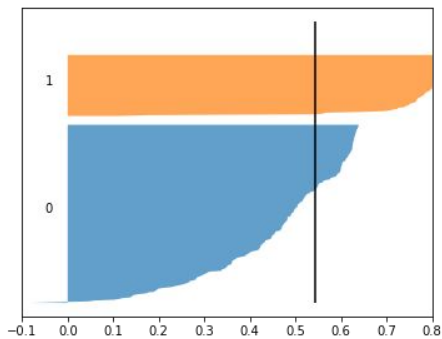
Silhouette

Neste exemplo, o gráfico de silhueta mostra que os valores de $n=[2, 7]$ tem alguns pontos negativos.



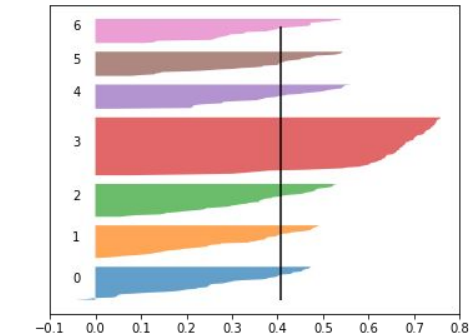
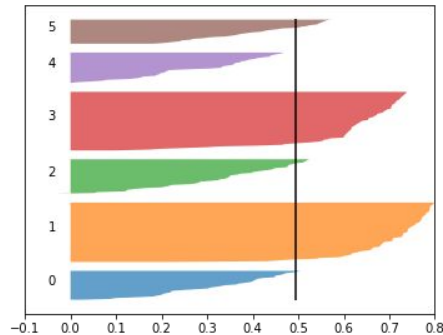
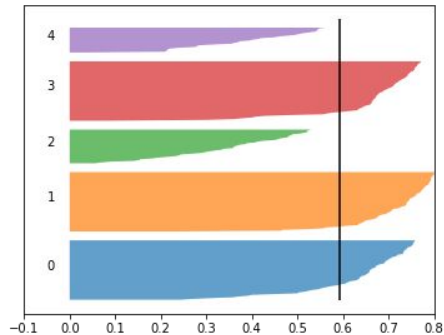
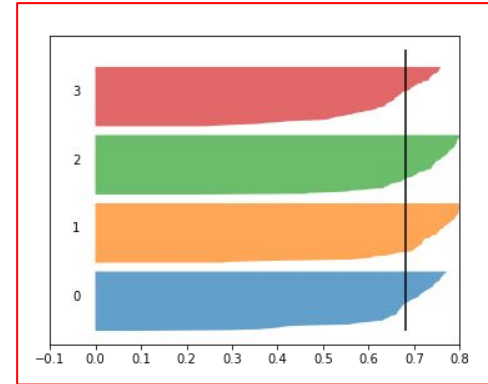
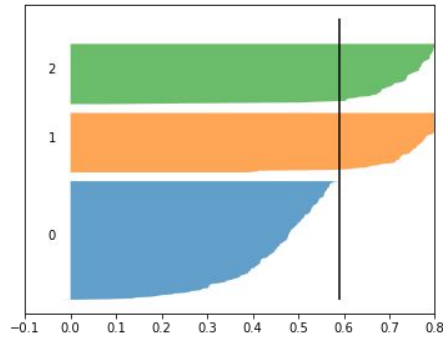
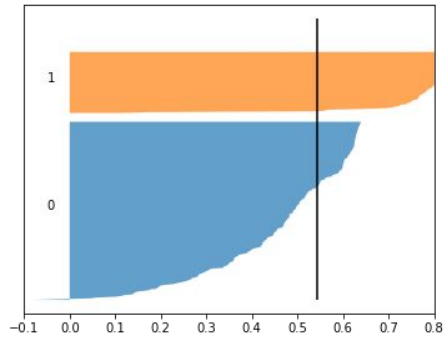
Silhouette

Se olharmos para o score médio, alguns cluster ficaram por baixo, por exemplo $n=[2, 5, 6]$. O maior score médio foi o 4.



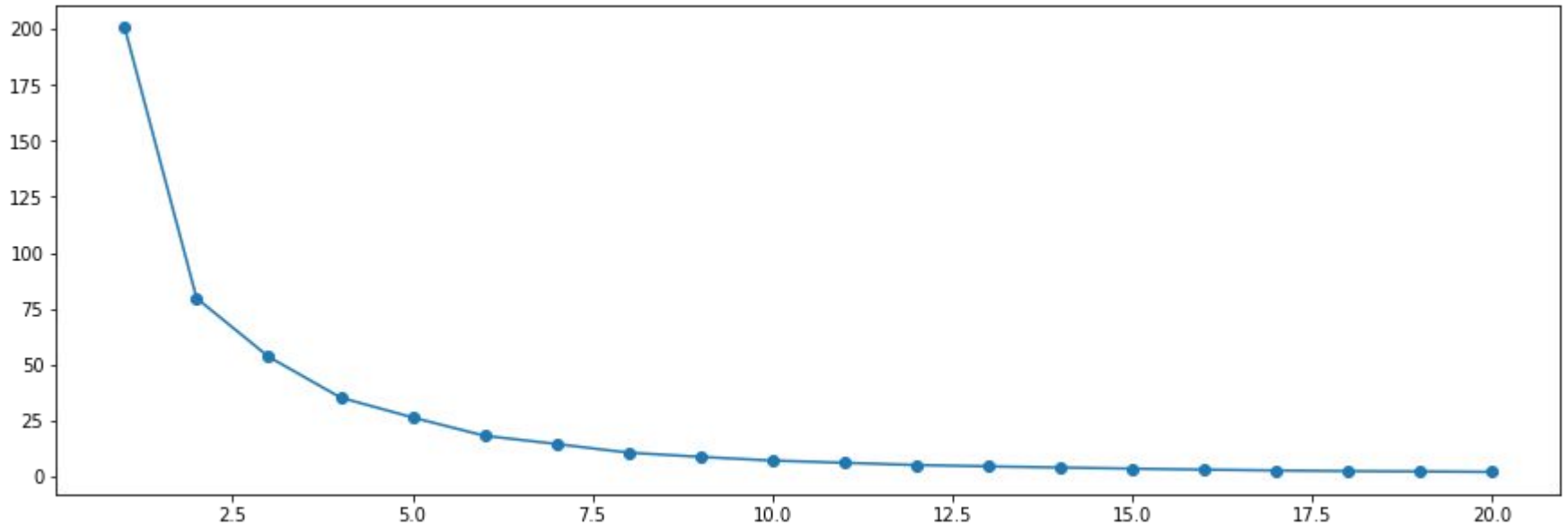
Silhouette

O cluster melhor equilibrado em relação ao tamanho seria 4.



Elbow

O objetivo é encontrar o “cotovelo da curva” de inercia.



Elbow

O que geralmente acontece ao aumentar a quantidade de clusters no k-means é que as diferenças entre clusters se tornam muito pequenas, e as diferenças das observações intra-clusters vão aumentando.

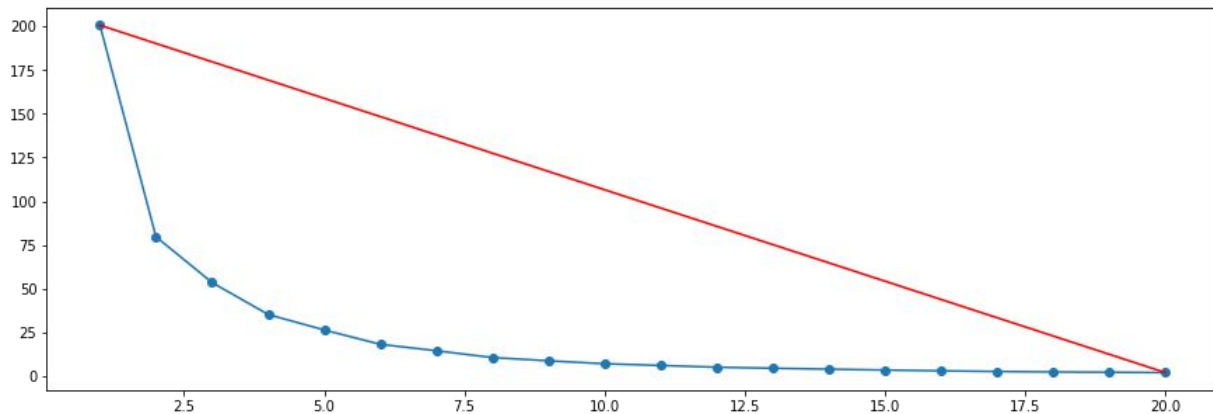
Então é preciso achar um equilíbrio em que as observações que formam cada agrupamento sejam o mais homogêneas possível e que os agrupamentos formados sejam o mais diferentes um dos outros.

Matematicamente falando, nós estamos buscando uma quantidade de agrupamentos em que a soma dos quadrados intra-clusters (ou do inglês within-clusters sum-of-squares, comumente abreviado para wcss) seja a menor possível, sendo zero o resultado ótimo, conhecido como **Inércia**:

$$L = \sum_{i=0}^n \min_{\mu_j \in C} \left(||x_i - \mu_j||^2 \right)$$

Elbow

O método utiliza uma linha reta e calcula a distância de cada ponto para esta linha, o ponto com maior distância ganha.

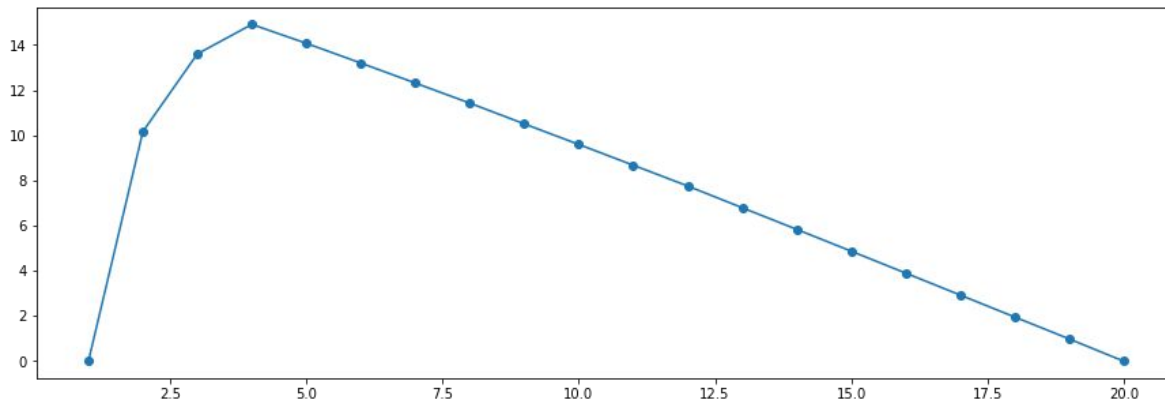


O equilíbrio entre maior homogeneidade dentro do cluster e a maior diferença entre clusters, é o ponto mais distante da curva:

$$\text{distance}(P_0, P_1, (x, y)) = \frac{|(y_1 - y_0)x - (x_1 - x_0)y + x_1y_0 - y_1x_0|}{\sqrt{(y_1 - y_0)^2 + (x_1 - x_0)^2}}$$

Elbow

As distâncias de cada ponto para a linha vermelha são:



Indicando $k=4$ como melhor candidato.

Pontos negativos

Pontos negativos do método de inércia:

1. A inércia é uma métrica que assume que seus clusters são convexos e isotrópicos, ou seja, se seus clusters são alongados ou de formato irregular essa é uma métrica ruim;
2. A inércia não é uma métrica normalizada, então se você tiver um espaço com muitas dimensões você provavelmente vai enfrentar a “maldição da dimensionalidade” já que as distâncias ficam infladas em espaços com muitas dimensões.

Spectral Clustering

O Spectral clustering utiliza a mesma ideia do SVM com kernel para transformar a matriz de features numa nova matriz de similaridades entre as amostras.

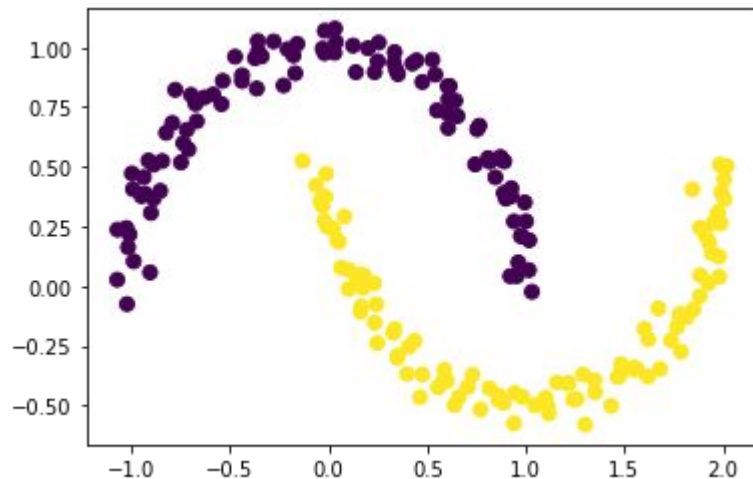
Utiliza a projeção do kernel RBF para projetar os dados em alta dimensão, onde uma separação linear é possível. Dessa forma, pode-se aplicar o k-means com função de decisão linear num espaço não linear.

A versão implementada no Scikit-Learn é “SpectralClustering”.

Ele usa o gráfico dos vizinhos mais próximos para calcular uma representação em altas dimensões dos dados, aplica a decomposição em autovetores e autovalores, e, em seguida, aplica o algoritmo k-means nos autovetores mais relevantes.

Spectral Clustering

Vemos que, após aplicar a transformação não linear do kernel, o k-means é capaz de encontrar os clusters mesmo quando as fronteiras entre os grupos são não lineares.



Atividade 1

Primeiro exemplo.

Tentaremos usar k-means identificar dígitos (8x8 pixels) semelhantes sem usar as informações do rótulo original

Isso pode ser semelhante a uma primeira etapa na extração de conhecimento de um novo conjunto de dados sobre o qual não temos nenhuma informação de rótulo a priori.

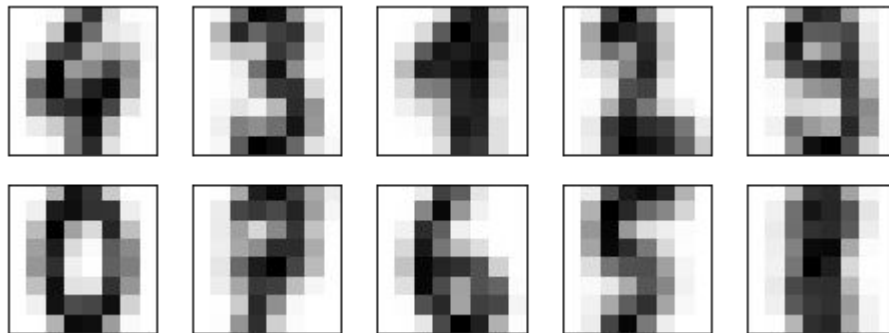
link:

Atividade 1

O resultado são 10 centróides em 64 dimensões.

Observe que os centróides são pontos de 64 dimensões e podem ser interpretados como o dígito "típico" dentro do cluster.

Vamos ver como eles são:

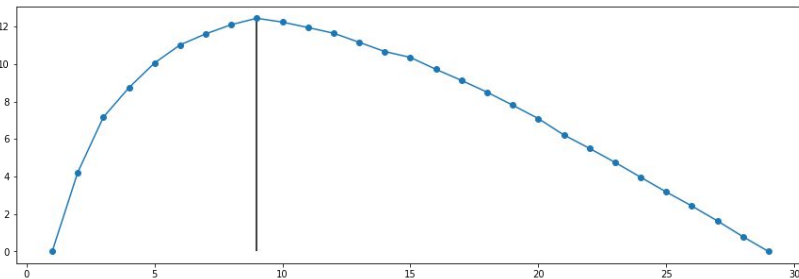
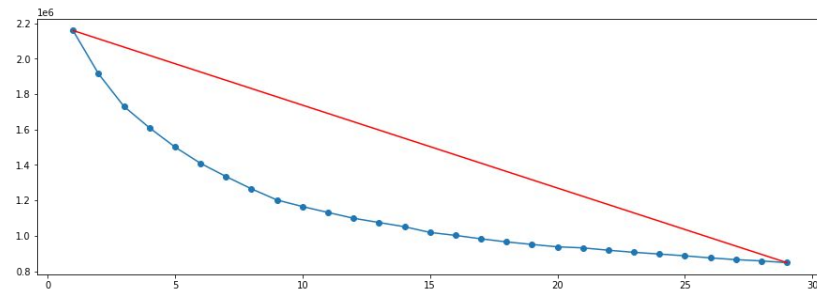
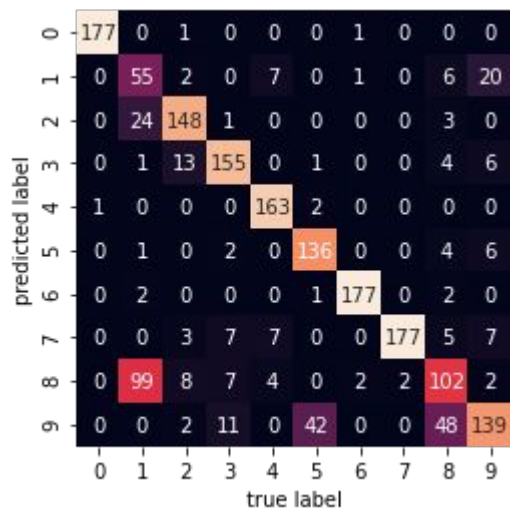


Vemos que mesmo sem os rótulos, “KMeans” é capaz de encontrar clusters cujos centros são dígitos reconhecíveis, talvez com exceção do 1 e 8.

Atividade 1

O ELbow sugere um 9 grupos.

Aproveitando que temos os rótulos reais, podemos quantificar a coerência dos membros dos clusters.



Atividade 2

Uma aplicação interessante de agrupamento é a compactação de cores em imagens.

Por exemplo, imagine que você tem uma imagem com milhões de cores.

Na maioria das imagens, um grande número de cores não será perceptível porque muitos dos pixels da imagem terão cores semelhantes ou até idênticas.



Atividade 2

Neste caso, podemos escolher um número baixo de k (por exemplo 16) e aplicar o k-means para agrupar pixels semelhantes.

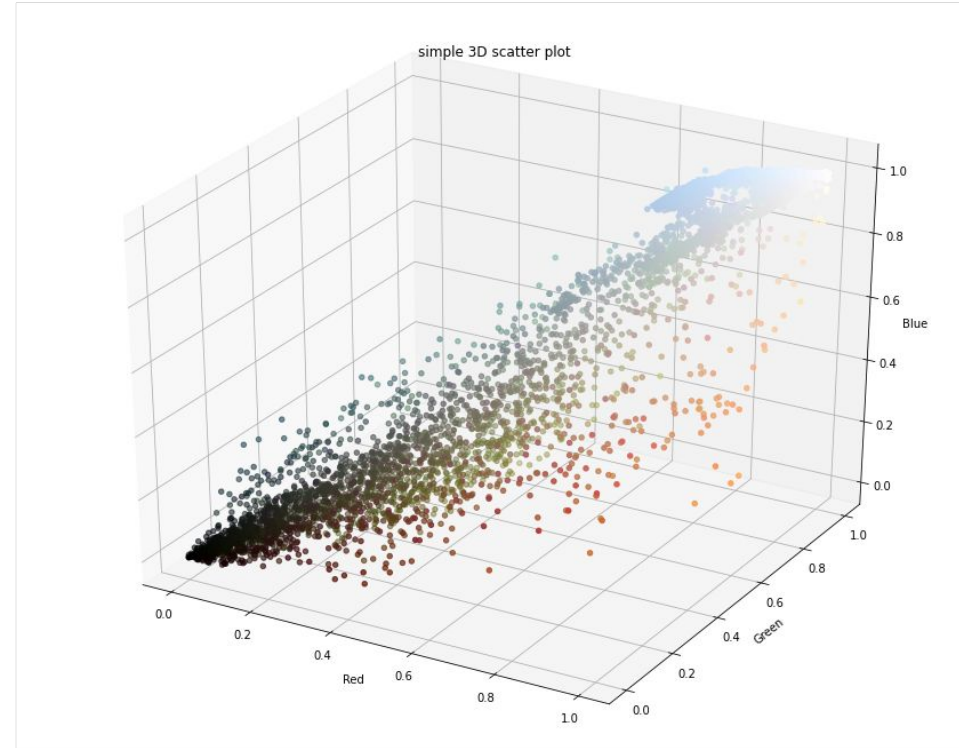


A imagem em si é armazenada em uma matriz tridimensional de tamanho (altura, largura, RGB), contendo contribuições vermelho / azul / verde como números inteiros de 0 a 255.

São 819840 pixels!

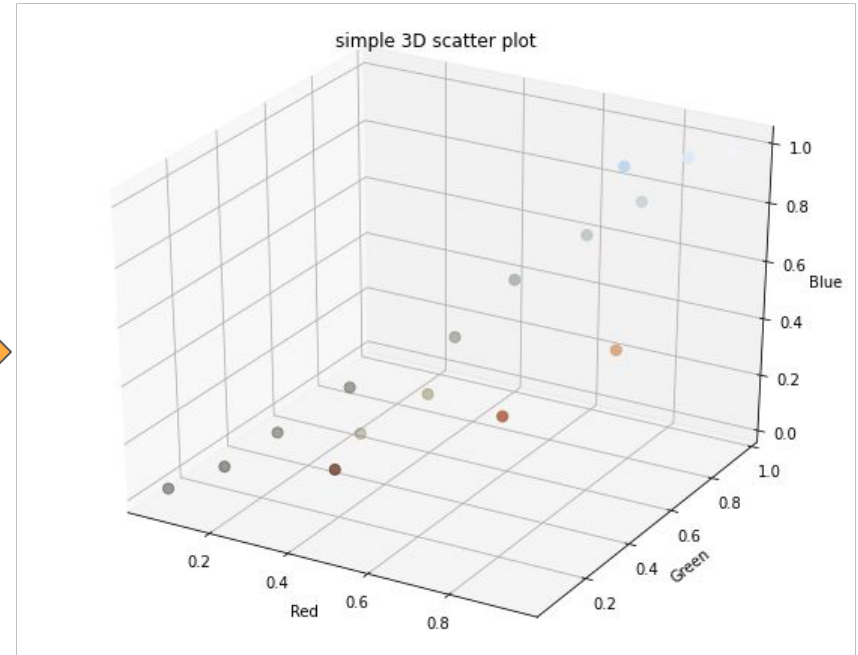
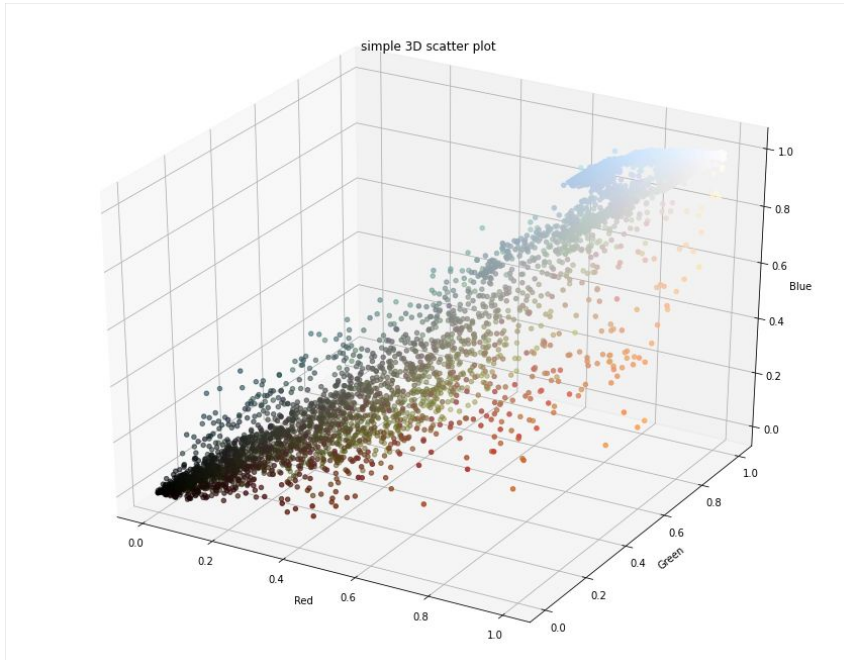
Atividade 2

Visualizando a distribuição de pixels pela cor. Um subconjunto de pixels escolhido aleatoriamente.



Atividade 2

Os novos centróides são



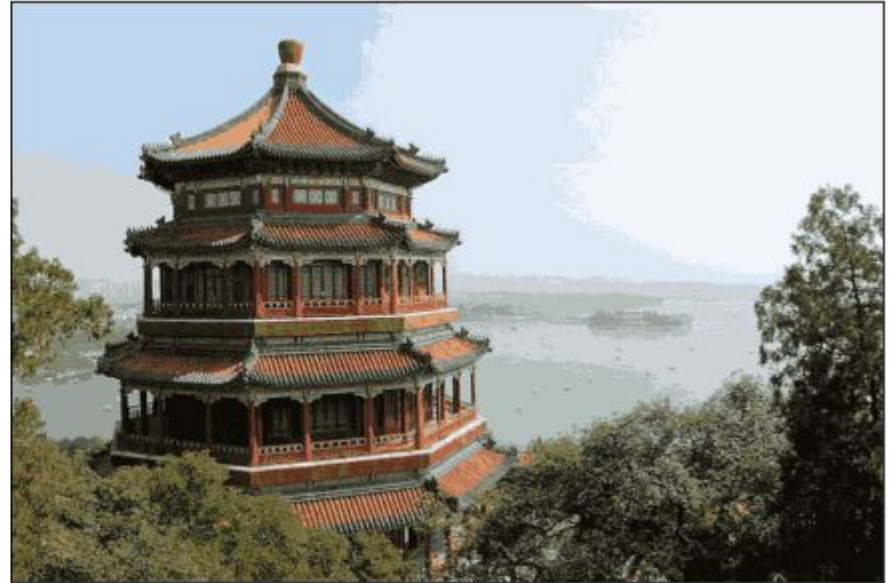
Atividade 2

Depois de mapear cada pixel para o valor do centróide do seu grupo temos como resultado a imagem com 16 cores!

Original Image



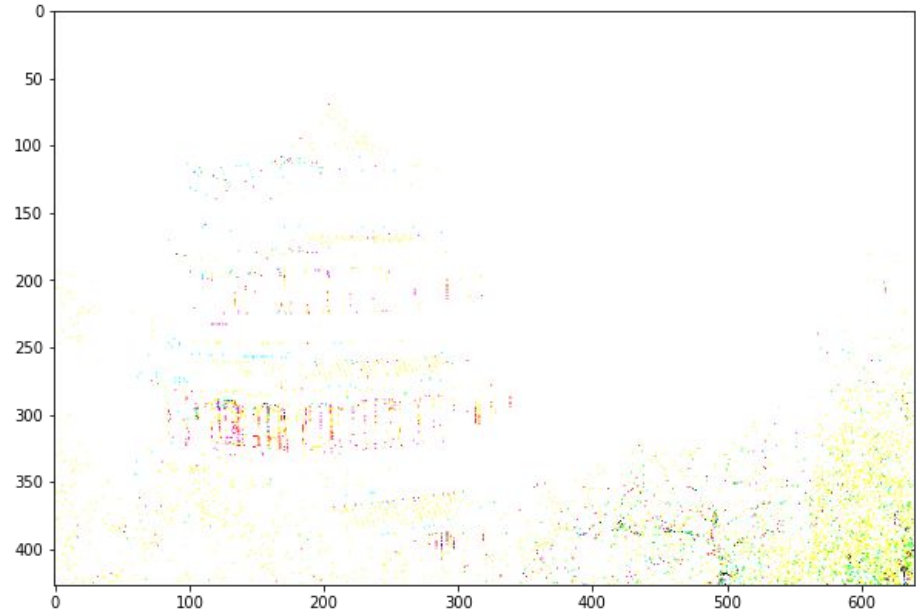
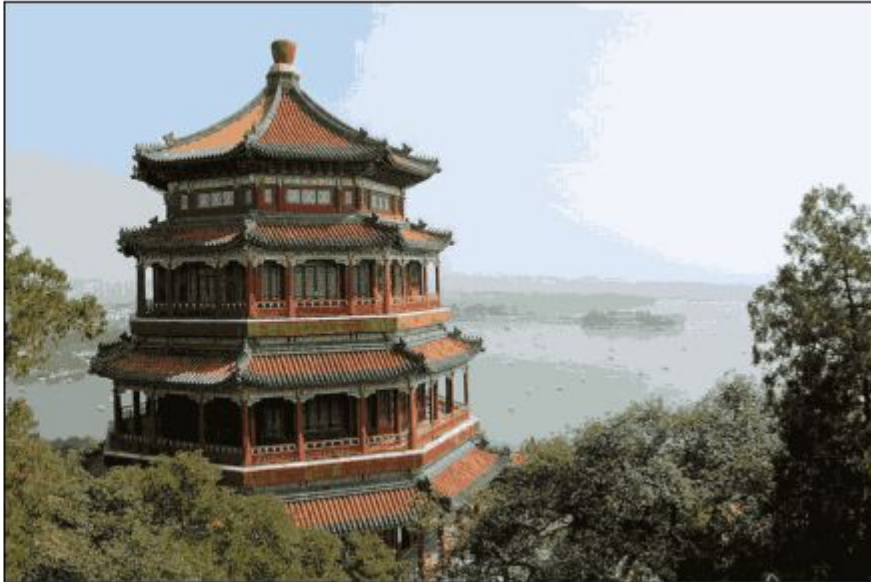
16-color Image



Atividade 2

Diferença entre as imagens. Podemos observar em quais regiões ou mais perda de informação de cores.

16-color Image



Atividade prática

Repetir a atividade de compressão de cores da imagem mas utilizando qualquer algoritmo de cluster do sklearn (<https://scikit-learn.org/stable/modules/clustering.html>).

A escolha do algoritmo deve ser sábia, ou seja, antes de escolher o algoritmo ver algum critério do funcionamento que seja interessante para o problema da imagem.

Gerar as imagens com a diferença dos pixels entre a imagem original e a nova compressão, e entre a nova compressão e o kMeans.

Discutir qual método é melhor e o por que.